

PROJET

Théorie des Langages et Compilation - Projet Flex
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2025/2026

Enseignant : Mme. Imane Lasri

Date : 11 mai 2026

Table des matières

Introduction à Flex	2
1 Exercice 1 : Suppression des espaces redondants	3
1.1 Énoncé	3
1.2 Implémentation	3
1.3 Code source	3
1.4 Résultat d'exécution	4
2 Exercice 2 : Reconnaissance des entités lexicales	4
2.1 Énoncé	4
2.2 Implémentation	4
2.3 Code source	5
2.4 Résultat d'exécution	6
Conclusion	6

Introduction à Flex

Flex est un outil de génération automatique d'analyseurs lexicaux. Un fichier Flex contient la description d'un analyseur lexical à générer. Cette description est exprimée à l'aide d'expressions régulières étendues et de code écrit en langage C ou en C++. Flex produit en sortie un fichier contenant le code C du futur analyseur lexical. Ce fichier porte généralement le nom `lex.yy.c`. La compilation de ce fichier à l'aide d'un compilateur C permet ensuite d'obtenir le programme exécutable de l'analyseur lexical.

Structure d'un fichier Flex

Un fichier `.l` est composé de trois sections séparées par `%%` :

1. **Section des déclarations** : inclusion de bibliothèques C et définition de macros.
2. **Section des règles** : association entre expressions régulières et actions C.
3. **Section du code utilisateur** : fonctions auxiliaires (`main`, `yywrap`, etc.).

Commandes de compilation

Pour compiler et exécuter un fichier Flex :

```
flex compil.l  
gcc -g -o nom_executable lex.yy.c  
./nom_executable
```

1 Exercice 1 : Suppression des espaces redondants

1.1 Énoncé

L'objectif de cet exercice est d'écrire un liseur Flex qui supprime les espaces redondants. La sortie du programme doit être le texte d'entrée dans lequel chaque séquence d'espaces est remplacée par un seul espace. Le liseur a ensuite été modifié afin que tous les espaces apparaissant en début de ligne soient supprimés.

1.2 Implémentation

Le fichier `ex1.1` contient les règles suivantes :

- **Règle 1** (`^\t*[ \t]+`) : supprime tout espace ou tabulation en début de ligne.
- **Règle 2** (`[ \t]+`) : remplace toute séquence d'espaces/tabulations par un seul espace.
- **Règle 3** (`\n`) : affiche les sauts de ligne tels quels.
- **Règle 4** (`.`) : affiche tout autre caractère tel quel.

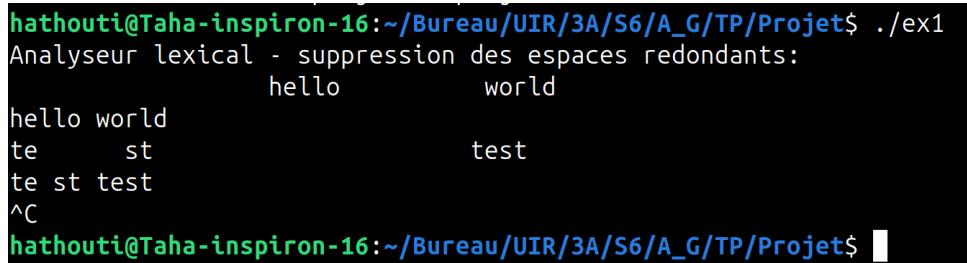
1.3 Code source

Listing 1 – `ex1.1` – Suppression des espaces redondants

```
1 %{
2 #include <stdio.h>
3 %}
4
5 %%
6
7 ^\t*[ \t]+      { /* supprime les espaces en debut de ligne */ }
8 [ \t]+         { printf(" "); }
9 \n             { printf("\n"); }
10 .             { printf("%s", yytext); }
11
12 %%
13
14 int yywrap() {
15     return 1;
16 }
17
18 int main() {
19     printf("Analyseur lexical - suppression des espaces
20           redondants:\n");
21     yylex();
22     return 0;
23 }
```

1.4 Résultat d'exécution

La capture d'écran ci-dessous montre le fichier `ex1.1` ouvert dans un éditeur ainsi que le résultat de l'exécution dans le terminal.



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/A_G/TP/Projet$ ./ex1
Analyseur lexical - suppression des espaces redondants:
hello world
te      st
te st test
^C
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/A_G/TP/Projet$
```

FIGURE 1 – Capture d'écran : fichier `ex1.1` et exécution dans le terminal

2 Exercice 2 : Reconnaissance des entités lexicales

2.1 Énoncé

L'objectif de cet exercice est d'écrire un liseur Flex capable de reconnaître les entités suivantes :

- **Nombres** : séquences de chiffres (ex. : 12, 42) ;
- **Mots-clés** : `begin` et `end` ;
- **Identificateurs** : séquences de lettres et de chiffres commençant par une lettre et ne correspondant pas à un mot-clé ;
- **Opérateurs** : `+`, `-`, `*` et `**`.

Les espaces, les tabulations et les retours à la ligne sont des séparateurs ignorés. Toute autre séquence de caractères qui ne constitue pas une entité valide est considérée comme une **erreur**.

Le liseur affiche à l'écran la liste des entités reconnues, par exemple : `nombre 12`, `identificateur abc`, `erreur #`, etc.

2.2 Implémentation

Deux définitions de macros sont utilisées :

- `nombre` : expression régulière `[0-9]+` — une ou plusieurs chiffres.
- `identif` : expression régulière `[a-zA-Z][a-zA-Z0-9]*` — lettre suivie de lettres/-chiffres.
- `separateur` : expression régulière `[\t\n]+` — espaces, tabulations, retours à la ligne.

L'ordre des règles est important : les mots-clés (`begin`, `end`) sont définis *avant* la règle des identificateurs pour être reconnus en priorité. L'opérateur `**` est aussi placé avant `*` pour que Flex applique la règle la plus longue en priorité.

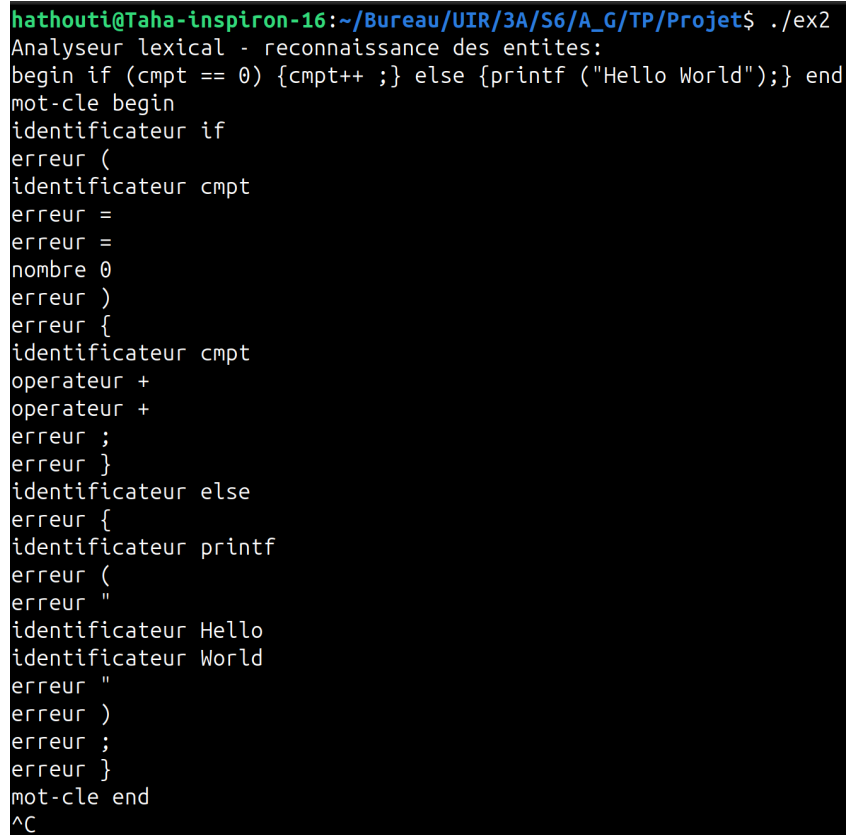
2.3 Code source

Listing 2 – ex2.1 – Reconnaissance des entités lexicales

```
1 %{
2 #include <stdio.h>
3 %}
4
5 nombre      [0-9]+
6 identif     [a-zA-Z][a-zA-Z0-9]*
7 separateur  [ \t\n]+
8
9 %%
10
11 {separateur}    { /* separateurs ignores */ }
12 "begin"         { printf("mot-cle begin\n"); }
13 "end"           { printf("mot-cle end\n"); }
14 "**"            { printf("opérateur **\n"); }
15 "+"            { printf("opérateur +\n"); }
16 "-"            { printf("opérateur -\n"); }
17 "*"            { printf("opérateur *\n"); }
18 {nombre}        { printf("nombre %s\n", yytext); }
19 {identif}       { printf("identificateur %s\n", yytext); }
20 .               { printf("erreur %s\n", yytext); }
21
22 %%
23
24 int yywrap() {
25     return 1;
26 }
27
28 int main() {
29     printf("Analyseur lexical - reconnaissance des entites:\n");
30     yylex();
31     return 0;
32 }
```

2.4 Résultat d'exécution

La capture d'écran ci-dessous montre le fichier `ex2.1` ouvert dans un éditeur ainsi que le résultat de l'analyse affichée dans le terminal.



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/A_G/TP/Projet$ ./ex2
Analyseur lexical - reconnaissance des entites:
begin if (cmt == 0) {cmt++ ;} else {printf ("Hello World");} end
mot-cle begin
identificateur if
erreur (
identificateur cmt
erreur =
erreur =
nombre 0
erreur )
erreur {
identificateur cmt
operateur +
operateur +
erreur ;
erreur }
identificateur else
erreur {
identificateur printf
erreur (
erreur "
identificateur Hello
identificateur World
erreur "
erreur )
erreur ;
erreur }
mot-cle end
^C
```

FIGURE 2 – Capture d'écran : fichier `ex2.1` et exécution dans le terminal

Conclusion

Ce projet a permis de mettre en pratique l'utilisation de l'outil Flex pour la génération d'analyseurs lexicaux. L'exercice 1 a illustré la manipulation des espaces dans un flux de texte, tandis que l'exercice 2 a montré comment reconnaître plusieurs catégories de tokens (mots-clés, identificateurs, nombres, opérateurs) en gérant les priorités entre règles.

L'ordre des règles dans un fichier Flex est crucial : Flex applique la règle dont le motif est le plus long (*maximal munch*), et en cas d'égalité de longueur, la première règle définie est choisie. C'est ce mécanisme qui permet de distinguer `begin` (mot-clé) d'un identificateur quelconque, ou `**` de deux `*` séparés.